Research Master's in Computer Science, Intelligent Pervasive Systems

Evaluating Coding Agent Quality

Application to Software Design Principles

Akram Ben Fekih

Supervised by Mrs. Maha Khemaja

ISSAT Sousse, University of Sousse · 2025/2026

Outline

1. Problem, state of the art and research questions
2. The five principles as measurable proxies
3. The before/after delta
4. Corpus: two languages, three agents
5. Pipeline and algorithms
6. Validation
7. Results per agent and per language
8. Findings, conclusions, limitations

Part 1

Problem, state of the art and research questions

Coding agents now submit real changes

- Claude Code, GitHub Copilot and OpenAI Codex take a task and open a pull request.
- The AIDev dataset records **7.69 million** agent pull requests in 957k repositories.
- They are evaluated mainly on whether tests pass (HumanEval, SWE-bench).
- Passing tests says nothing about **design**: whether the next developer can change the code.

about half

of SWE-bench Verified patches that pass the tests would not be merged by maintainers (METR, 2026)

77%

of long-horizon agent trajectories show growing structural erosion (SlopCodeBench, 2026)

State of the art: what agents do to code

- Passing the tests is not the same as being accepted: about **half** of passing SWE-bench Verified patches would not be merged, **24 points** below the automated score.
  Whitfill et al., METR, 2026
- Functional correctness hides structural debt: Cursor projects of **16,965** lines reach **91%** correctness, yet carry **1,305** CodeScene and **3,193** SonarQube design issues.
  Kashif et al., 2026
- Structure decays over long horizons: erosion grows in **77%** of trajectories, and agent code is **2.3**× more verbose than human open-source Python.
  SlopCodeBench, Orlanski et al., 2026
- AI defects have a shape, broken abstractions, duplication, over-engineering, and a **Volume-Quality Inverse Law**: more code generated, more structure decayed.
  Zhu, Yuecai et al., 2026
- Agents are poor at *repairing* structure: **4.3%** success on dependency control, **15.2%** on responsibility decomposition.
  Zhu, Haichao et al., NITR, 2026

State of the art: measurement and attribution

- A language model can detect SOLID violations. **SRP is easiest, DIP hardest**, and accuracy falls as complexity rises. This is the technique our judge uses.

  Pehlivan et al., 2025

- Differences between agents **largely vanish once normalised for churn**, and a merge does not guarantee structural quality.

  Cynthia et al., 2026

- After a merge, **humans do almost all the maintenance** on agent-written files, so maintainability is a human cost.

  Sawada et al., 2026

- The **harness** around the model, its tools and context management, affects quality independently of the LLM.

  Ben Sghaier et al., 2026

- "Vibe architecting": agents make implicit architectural choices from prompt nuance, giving **141 to 827 lines** for one and the same task.

  Konrad et al., 2026

The gap, and what this thesis does

What existing work does

- Benchmarks on functions or small task sets
- LLMs detecting SOLID violations in examples
- Smell counts on merged code only
- Metrics assumed to capture a principle

What this thesis does

- Real pull requests from three agents
- All five SOLID principles, one detector each
- Merged, rejected *and* open requests
- Every detector's precision and recall measured
- The whole study repeated in a second language

Unit of analysis: one pull request. Object of measurement: what that pull request did to the design of the code it touched.

Research questions

RQ1 **Agents and outcomes.** Do agents differ in SOLID adherence, and does merged code differ from code that was not merged?

RQ2 **Measurability.** Can SOLID be measured with deterministic detectors, and how well do they agree with a reading judge and with published analysers?

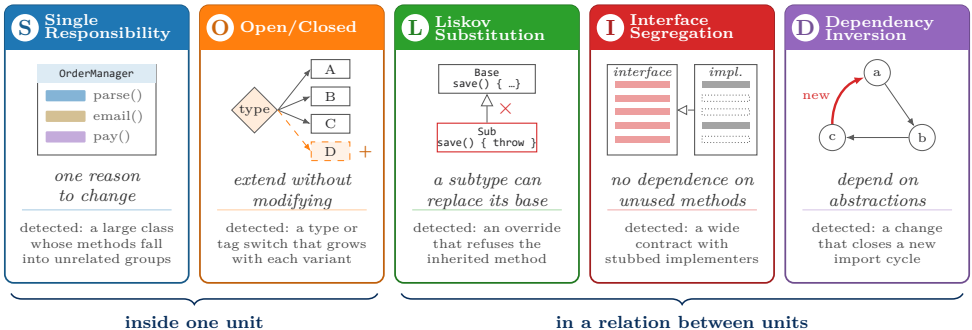
RQ3 **Locus of risk.** Do design problems lie inside single units, or in relations between units?

RQ4 **Generalisation.** Which of these answers hold again in Python?

Part 2

The five principles as measurable proxies

From principle to detector



Each principle is a statement about intent. A detector can only see the structure that a violation usually leaves behind, so every result is first a statement about the detector.

SRP: God Class rubric with a cohesion suppressor

Six size and complexity metrics; one *hit* per threshold reached:

$$h = \#\{m : \ \text{metric}_m \geq \tau_m\}$$

Cohesion, LCOM4 over the method graph (edge = shared field or a call), constructors excluded:

$$\text{LCOM} = \frac{c - 1}{n - 1}$$

Conceptual cohesion over TF-IDF vectors of method token bags:

$$C3 = \underset{i<j}{\text{mean}} \cos(v_i, v_j)$$

$$\text{flag} = (h \geq 2) \ \wedge \ \neg\,(C3 \geq 0.4 \wedge h < 3)$$

Threshold	Java	Python
class lines	362	250
weighted methods	45	36
methods	20	10
method complexity	11	12
parameters	5	7
LCOM	0.86	0.8

Each value is the 90th percentile of that corpus, fixed before any label existed.

0.350 / 0.110
precision, Java / Python

OCP: dispatch width with a closed-set suppressor

A dispatch chain is an `if/else if` ladder or a `switch` branching on one subject. With b branches:

$$\text{smell} = (b \geq \tau) \ \wedge \ \neg\,\mathrm{closed}(\text{subject})$$

closed holds when the subject's resolved type is an enumeration: a client cannot add a value.

A pull request is flagged when it **creates** or **widens** such a chain.

τ	Java	Python
type chains	3	3
tag / value chains	8	5

0.330 / 0.390
precision, Java / Python

Width does not predict truth: precision stays between 0.270 and 0.400 across every band. The sets that are closed in practice are fixed by specifications, not by the language.

LSP: override compared with its nearest ancestor

For each pair (subclass method, nearest ancestor method with the same signature and a real body):

$$\text{violation} = \exists\, s \in \text{signals}(\textit{pair}) : \; s \in S$$

Java S = refused bequest, strengthened precondition, weakened postcondition.

Python S adds what no compiler checks: parameter removed, required parameter added, parameter type narrowed, return type widened.

Java signal	precision
refused bequest	31/31
weakened postcondition	6/36
strengthened precondition	3/33

The shipped Java flag keeps refused bequest only: 64 of 64 across two samples.

0.420 / 0.510
precision of the full signal set, Java / Python

ISP: contract width and implementer stubs

A contract is an interface or abstract class with at least one abstract member and at least one implementer inside the repository. Width w = abstract members; for each implementer,

$$s = \frac{\#\text{stubbed members}}{\#\text{members it provides}}$$

Java: contract fat if $w \geq 13$; forced stub if stubs ≥ 2 and $s \geq 0.5$. Flag if the request creates a fat contract, widens one, or leaves a forced stub.

Python: flag if at least **two** of: $w > 7$; mean implementer coverage < 0.75 over ≥ 2 implementers; an implementer with forced stubs.

Members	judged	precision
0–17	43	0.116
18–29	28	0.071
30–49	14	0.786
50–99	10	0.900
≥100	5	1.000

Width does separate violations here, which is why 13 was too low a bar.

0.335 / 0.425
precision, Java / Python

DIP: new cycles in the import graph

Build the internal import graph before and after the change and take strongly connected components with Tarjan's algorithm. With $P(G)$ the set of module pairs inside one component:

$$\text{new} = P(G_{\text{after}}) \setminus P(G_{\text{before}})$$

$$\text{flag} = \exists (u, v) \in \text{new} : \; u \text{ or } v \text{ was changed}$$

Also computed, as description only: $I = \frac{C_e}{C_a + C_e}$, $A = \frac{N_a}{N_c}$, $D = |A + I - 1|$.

0.892 / 1.000

precision, Java / Python

Java precision is exact: all 195 flagged pull requests were judged. In Python, 100 of 173 were judged and all were correct.

The distance D was tested as a change signal and confirmed **none** of 17 flags, so it describes packages and never raises a flag.

Calibration: same rules, two languages

Principle	Java	Python
SRP	six metrics, $\geq$2 hits; 362 / 45 / 20 / 11 / 5 / 0.86	same rubric; 250 / 36 / 10 / 12 / 7 / 0.8
OCP	type $\geq$3, tag $\geq$8 branches	type $\geq$3, value $\geq$5 branches
LSP	three behavioural signals	five signals, adding what a compiler would reject
ISP	contract $\geq$13 members, or a forced stub	at least two of three signals, width alone never flags
DIP	new cyclic pairs through a touched file	new cyclic pairs through a touched module

Every threshold is the 90th percentile of its own corpus, set **before** any label existed; the result files were then deleted and the pass re-run, so no flag predates its threshold.

Part 3

The before/after delta

What is charged to the pull request

Step 1

recover two source trees

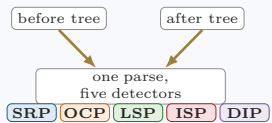


tried in order
1 merge commit (merged only)
2 the pull request's commits
3 pull ref and its merge base
4 base and head from the API

the method reached is recorded, and outcomes are compared within it

Step 2

run the same detector twice



each detector returns the set of violations it finds in that tree

Step 3

charge only the difference

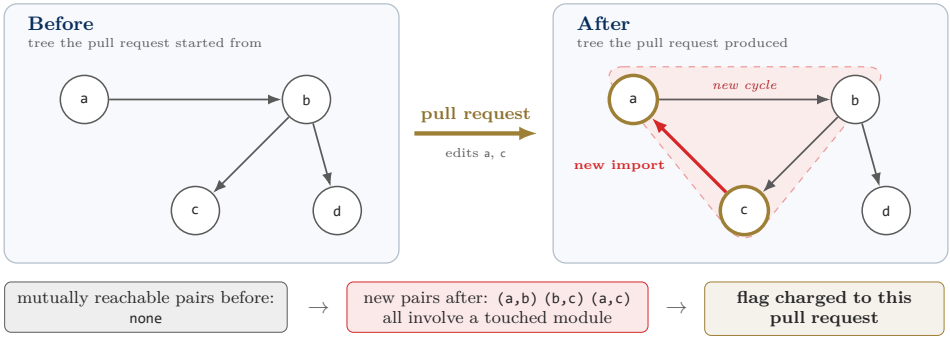


charged to the pull request: violations in **after** but not in **before**, when one end is a file the request changed

a new cycle, a newly widened contract, a newly refused override

Single Responsibility is not a delta: it is scored on the after tree, and classes the agent authored are reported separately from existing classes it edited.

The delta in the data



Attribution rule for relations: a violation is charged when **either** end is in a changed file, so an agent that widens a base class is charged for the subclasses it breaks in files it never opened.

Part 4

Corpus, pipeline and algorithms

Corpus: two languages, three agents, three outcomes

Java	merged	rejected	open
Claude Code	970	485	283
GitHub Copilot	2,544	1,272	364
OpenAI Codex	630	315	93
Python			
Claude Code	940	540	182
GitHub Copilot	885	407	178
OpenAI Codex	1,069	549	196

Java rows are the curated corpus (6,956), Python rows the analysable one (4,946).

4,245 / 4,946

analysable pull requests, Java / Python

- $\geq$25 stars, permissive licence
- balanced on agent and outcome
- Python also stratified by repository: no repository gives more than 0.4% of a cell

Attrition and how the two trees were recovered

Status	Java	Python
analysable	4,245	4,946
no production file touched	2,540	1,816
no module resolved	n/a	232
tree too large	150	14
history gone	18	2
no distinct before tree	3	40
curated	6,956	7,050

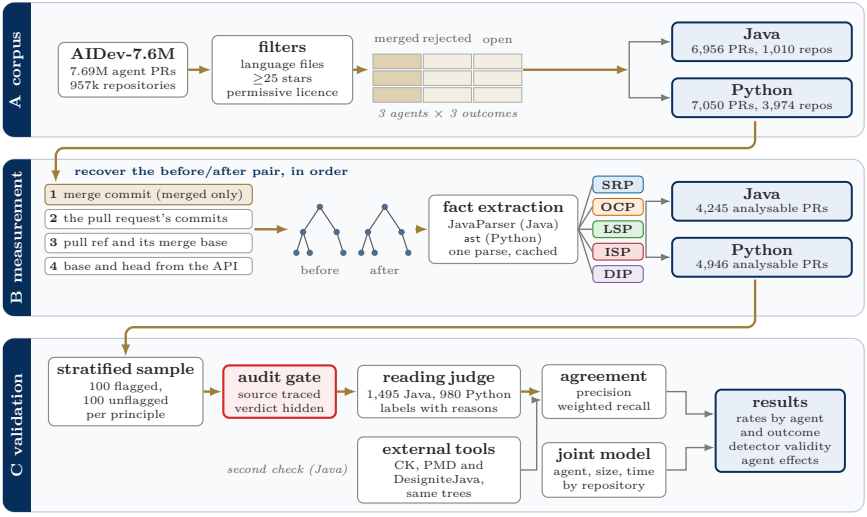
Recovery method by outcome (Java):

	1	2	3	4
merged	1,052	425	665	431
open	5	209	250	31
rejected	0	646	482	49

Method 1 is the merge commit, available to merged requests only. That is why every merged-versus-unmerged comparison is made **within the same method**.

7.0% of Python requests lost their source entirely: the branch was deleted.

Full workflow



Algorithms: from the dataset to the facts

Phase	Algorithm	Why it is needed
Curation	seeded stratified draw, balanced on agent and outcome, repository-stratified in Python	one agent would dominate any raw aggregate
Tree recovery	four fallbacks, `git archive`, cache keyed by content hash	branches are deleted, especially for closed requests
Java facts	JavaParser with symbol solving, source roots inferred from `package` lines	without resolution two classes named `Node` merge into one
Python facts	standard library `ast`, imports resolved by module path	no compiler and no type information is available

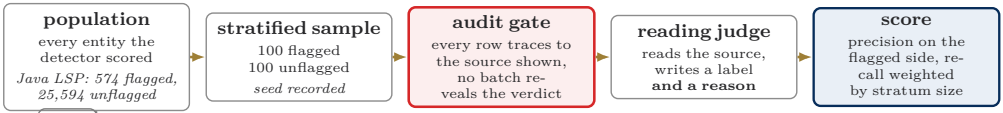
Algorithms: detectors and statistics

Phase	Algorithm	Why it is needed
SRP	union-find for LCOM4, TF-IDF cosine for C3	size alone cannot separate large from incoherent
OCP	dispatch chains grouped by subject, enum resolution	a switch over a fixed set is not a violation
LSP	nearest ancestor with a real body, over the resolved hierarchy	an abstract base has no behaviour to refuse
ISP	contract to implementer index over the hierarchy	inherited members are part of the contract
DIP	Tarjan strongly connected components	a cycle is a property of the whole graph
Sampling	seeded stratified draw after entity deduplication	classes touched by many requests would dominate
Scoring	Wilson intervals, stratum-weighted recall	the sample is half flagged, the corpus is not
Model	GEE, exchangeable within repository, cluster-robust errors	requests from one repository are not independent

Part 5

Validation

How a detector is validated



holdout (LSP, ISP)

a second sample, new seed, excluding every row judged before, judged before the two rounds are compared. A rule derived from false positives is only trusted after this.

census (DIP in Java, ISP in Python)

when a detector flags few entities, every flag is judged: 195 and 80. Precision is then exact, with no sampling interval.

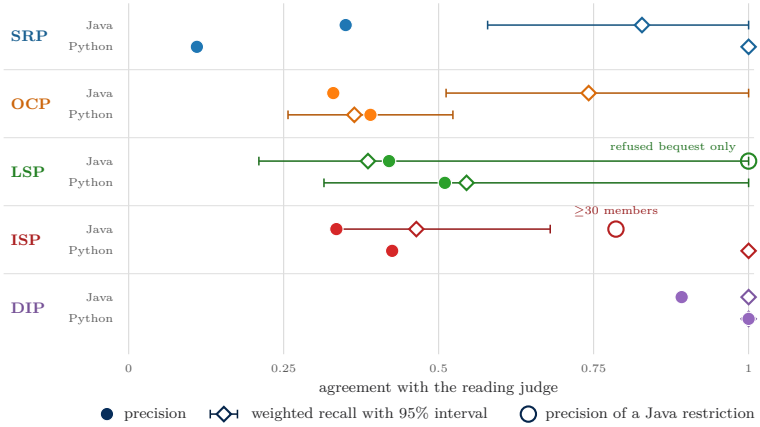
why recall is weighted

the sample is half flagged, the corpus is not:

$$R = \frac{TP \cdot N_f/n_f}{TP \cdot N_f/n_f + FN \cdot N_u/n_u}$$

N = population, n = sample, f = flagged, u = unflagged. Unweighted, Java LSP would read 0.966 instead of 0.386.

Validation results



2,475 entities judged, each with a written reason. Dependency Inversion is precise in both languages; Single Responsibility, Open/Closed and Interface Segregation are screens.

What the judge's disagreements are made of

Why flags fail

- **OCP:** the set is fixed by a specification, not by the language (JDBC codes, protobuf types, Unicode blocks, generated parser tables)
- **Python SRP:** classes whose size a framework imposes (model heads, home-automation entities, training callbacks)
- **ISP in Python:** test doubles and unfinished scaffolds that stub on purpose

What the detectors miss

- The three Liskov violations found in the unflagged sample are *silent*: an override that throws for every argument, one with an empty body, one that assigns `null`
- Structural rules catch refusals that are declared, not refusals that are quiet

89.7%

agreement when 174 Java entities were judged twice

Two refinements, and the test that separates them

Refinement	Derivation sample	Java holdout	Python, unchanged rule
LSP, refused bequest only	1.000 (31/31)	**1.000** (33/33)	**0.857** (6/7)
ISP, $\geq$30 members	0.862 (25/29)	0.704 (19/27)	0.200 (1/5)

- Refused bequest is **categorical**: a body either is only a throw, or it is not.
- A width of 30 was chosen where the false positives stopped, and absorbed the noise of that sample.
- Both rules were fixed on Java data before any Python label existed, so Python is a second, out-of-language test.
- Lesson: a rule that names a *shape* of code transfers; a tuned number does not.

Second check: three published analysers (Java)

God Class detectors, 82,553 types	Cohen's κ
ours vs PMD `GodClass`	0.574
ours vs DesigniteJava	0.531
PMD vs DesigniteJava	0.384
Open/Closed, 63,326 types	
ours vs Designite type-conditional	0.122

Rank correlation with CK	ρ
weighted methods	0.977
lines of code	0.944
cohesion (LCOM1)	0.288

- 150 random repositories, tools at their own default thresholds
- Our rubric agrees with each tool **more** than the two tools agree with each other
- 68.0% of our flags are confirmed by at least one of them
- Cohesion disagrees everywhere, including CK with itself (0.490)
- **Open/Closed is the exception**: it fires 3× as often as Designite's smells and shares only 76 types, so it rests on the judge alone

Part 6

Results

Flag rate by agent, both languages pooled, with the average over repositories in brackets

Principle	Java Claude	Java Copilot	Java Codex	Python Claude	Python Copilot	Python Codex
SRP (authored classes)	9.5 (8.4)	5.1 (3.5)	6.8 (7.4)	9.6 (11.0)	6.6 (10.3)	8.9 (8.1)
OCP	10.6 (10.1)	6.4 (3.0)	2.1 (2.2)	9.7 (9.6)	3.5 (3.1)	5.7 (6.6)
LSP	0.7 (0.3)	0.3 (0.1)	0.0 (0.0)	1.6 (1.2)	0.5 (0.3)	0.8 (0.8)
ISP	8.7 (6.0)	6.7 (4.0)	2.9 (5.0)	1.3 (1.0)	0.4 (0.4)	0.3 (0.1)
DIP	6.5 (5.1)	4.1 (2.8)	3.2 (5.4)	5.8 (5.6)	2.0 (1.5)	2.6 (2.3)

Read this only with the next two slides: the agents do not write changes of the same size, and most of this ordering follows from that.

Is the instrument fair to each agent? precision by agent: true positives over flags judged

	Java			Python		
Principle	**Claude**	**Copilot**	**Codex**	**Claude**	**Copilot**	**Codex**
SRP	18/55	10/38	7/7	6/46	2/26	3/28
OCP	23/68	7/25	3/7	20/45	7/19	12/36
LSP	53/106	22/68	9/26	29/73	11/15	11/12
ISP	35/83	26/93	6/24	23/48	6/17	5/15
DIP	57/72	94/98	23/25	53/53	16/16	31/31

- Claude Code's flags are confirmed **more** often on four of five Java principles, so its higher rates are not produced by the detector.
- Dependency Inversion is the exception in Java (0.792 against 0.959).
- The judge is a Claude model and one agent is Claude Code: leniency would show up as *fewer* of its flags confirmed, and it does not.

Change size explains most of the difference

Mean files per pull request:

Java: Claude **46.4**, Copilot **18.5**, Codex **10.9**
Python: Claude **16.5**, Copilot **7.7**, Codex **9.6**

	Files	Claude	Copilot	Codex
OCP	1–3	1.3%	**1.9%**	0.4%
	4–10	7.0%	3.5%	1.9%
	11+	**31.3%**	19.3%	8.9%
DIP	1–3	0.4%	0.5%	**0.8%**
	4–10	1.8%	2.4%	**3.7%**
	11+	**22.3%**	13.7%	12.1%

- The ordering holds only among the largest changes.
- Among changes of three files or fewer, Copilot leads on Open/Closed and Codex on Dependency Inversion.
- A delta detector has more chances to fire when more code changes, so size has to be adjusted for, not just described.

The joint model

$$\operatorname{logit} \Pr(\text{flag}) = \beta_0 + \beta_{\text{agent}} + \beta_1 \log(1 + \text{files}) + \beta_2 \,\text{months}$$

estimated by GEE, exchangeable within repository, cluster-robust errors; Copilot is the reference.

	Java	Python
odds per doubling of files	1.88–1.95	2.01–2.18
Claude Code, OCP	2.22	2.09
95% interval	1.21–4.07	1.44–3.04
p	0.010	0.0001
time, per month	≈1	≈1

- **Change size dominates** every principle in both languages.
- **One agent difference replicates**: Claude Code on Open/Closed.
- Six agent coefficients are tested in Java and ten in Python. Java alone would not pass a Bonferroni correction; Python does, so the claim rests on the replication.
- Python's import-cycle effect (1.88, $p = 0.0096$) does not pass and is reported as unconfirmed.

Merged against unmerged code

Java, same recovery method

Stratum	n	OCP	DIP
merged	1,521	8.3%	4.6%
unreviewed	1,006	7.5%	5.6%
reviewed, closed	135	12.6%	12.6%

Merged code is indistinguishable from code closed without review.

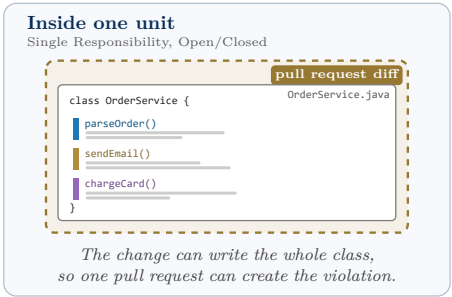
Python, same recovery method

Stratum	n	OCP	DIP
merged	1,734	4.9%	3.1%
unreviewed	1,300	9.1%	4.5%
reviewed, closed	163	7.4%	8.6%

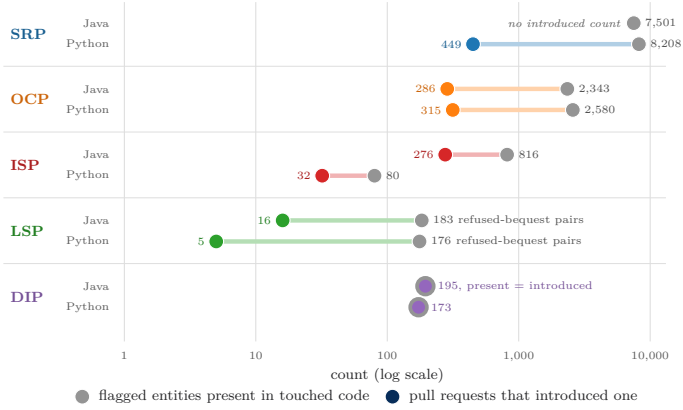
Merged code is cleaner: unmerged odds 1.57 (OCP) and 2.43 (LSP) after size and agent.

Only 171 of 1,177 unmerged Java requests and 196 of 1,496 Python ones were ever reviewed, so neither result shows what reviewers select on.

Where the problems live



Most violations were there before the change



Refused bequest: 183 present against 16 introducing pull requests in Java, 176 against 5 in Python. After correcting for precision, 0.4% to 4.1% of Java requests introduce a violation.

Agent-written classes are flagged less than the code they edit



Pooled: 7.7% against 13.5% in Java, 8.9% against 12.1% in Python. New classes are also smaller and younger, so this describes the corpus; it does not rank agents against people.

Do agents repair design, or damage it? cycles in the touched code, before against after

	Java (4,245 PRs)	Python (4,946 PRs)
more cycles after	188 (4.4%)	58 (1.2%)
fewer cycles after	20 (0.5%)	32 (0.6%)
unchanged	4,037 (95.1%)	4,856 (98.2%)
total cycles, before → after	8,028 → 8,398	1,485 → 1,516

Agents add cycles more often than they remove them, by about nine to one in Java. The amounts are small next to the 8,028 already present, and this is the only principle where a repair can be counted, because only Dependency Inversion stores a before count.

Part 7

Findings and conclusions

RQ4: what holds in both languages

Result	Java	Python
Change size dominates	OR 1.88–1.95	**holds** 2.01–2.18
Claude Code on Open/Closed	2.22 (1.21–4.07)	**holds** 2.09 (1.44–3.04)
Present far exceeds introduced	183 vs 16	**holds** 176 vs 5
Authored flagged less than edited	7.7% vs 13.5%	**holds** 8.9% vs 12.1%
Shape rule transfers, number does not	1.000 vs 0.704	**holds** 0.857 vs 0.200
Compiler removes a family of LSP faults	inferred from Java	**confirmed** 46 of 51
Merged equals unreviewed	yes	**does not hold**
SRP screen precision	0.350	**drops** to 0.110

Findings

1. What a compiler enforces decides which violations can exist: 46 of 51 confirmed Python Liskov violations are faults `javac` rejects.
2. Width predicts Interface Segregation violations and not Open/Closed ones, because one principle is about size and the other about whether a set can grow.
3. The closed sets that matter are fixed by specifications, not by the language.
4. Most flagged problems in the touched code predate the change.
5. Classes the agents wrote are flagged less often than the classes they edit.
6. Merged code is not a biased sample of unreviewed agent code in Java, but it is cleaner than abandoned code in Python.

Against the state of the art: what we confirm

Earlier finding	What we measured	Outcome
Volume-Quality Inverse Law *(Zhu, Yuecai)*	Each doubling of files touched multiplies the odds of a flag by **1.88–1.95** (Java) and **2.01–2.18** (Python), on every modelled principle	**Confirmed.** Our strongest result, in both languages
Agent differences vanish after churn normalisation *(Cynthia et al.)*	After adjusting for files, repository and time, **only Claude Code on Open/Closed** survives: 2.22 Java, 2.09 Python	**Confirmed**, with one exception that replicates
Passing tests $\neq$ a change a maintainer accepts *(Whitfill et al.)*	Merged Java pull requests are **structurally indistinguishable** from those closed without review	**Consistent.** Merge status carries little structural information
91% correct, yet thousands of design issues *(Kashif et al.)*	Violations *already present* outnumber those *introduced* by about **11:1** (Java) and **35:1** (Python)	**Consistent**, though they build new projects and we edit existing ones

Against the state of the art: where we differ

Earlier finding	What we measured	Outcome
SRP easiest to detect, DIP hardest *(Pehlivan et al.)*	For *structural* detectors the order reverses: DIP is the most precise (**0.892** Java, **1.000** Python), SRP the least (**0.350**, **0.110**)	**Reversed**, because a model reads examples while a static rule computes a whole-repository graph
Agents succeed at only 4.3% of dependency-control tasks *(Zhu, Haichao, NITR)*	Agents **seldom introduce** dependency or responsibility violations into existing repositories	**Different question**: that benchmark asks agents to *repair* structure, we ask whether they *damage* it
Erosion grows across 77% of long-horizon trajectories *(SlopCodeBench)*	The time coefficient is **inert** for every delta principle, in both languages	**Not tested**: our unit is one pull request, not a trajectory of successive edits

Taken together with Pehlivan: a graph-based detector and a reading model fail on opposite principles, so combining them beats choosing one.

Contributions and practical use

Contributions

- A validated SOLID instrument over real agent pull requests, all five principles
- A corpus with merged, rejected and open requests
- A map of which principles a rule measures and which it only screens
- Evidence in two languages on where design risk sits
- A replication with the same agents in a second language

For a review check

- Add **refused bequest**: 64/64 in Java, 6/7 in Python
- In dynamic languages, also check that an override still accepts the base's calls
- Do not rely on interface-width thresholds
- Treat SRP and OCP flags as prompts for discussion, not as blockers
- Structural checks cannot see relation-level damage: that needs repository history

Limitations

- SOLID adherence is not measured design quality or cost; nothing here observes an outcome
- Labels come from an LLM judge of the same family as one agent; 89.7% self-agreement bounds every figure
- No independent-tool check for OCP, ISP, or for Python at all
- Agents chose their own tasks: these are associations, not causes
- Few reviewed rejections (171 Java, 196 Python)
- One snapshot per pull request: slow erosion is invisible
- Both corpora come from one dataset; 7.0% of Python source is gone
- Small counts for the relation-level principles, so their intervals are wide
- Python Interface Segregation is defined differently, so raw rates are not comparable

Conclusion and future work

Most design problems in the code these agents touch were there before them, and how much an agent changes matters more than which agent it is. Both results hold in Java and in Python.

Next steps

- Recognise specification-fixed sets for Open/Closed, and a churn signal for what is not closed
- An Open/Closed proxy that agrees with published smells ($\kappa = 0.122$ today)
- A framework-aware responsibility screen for Python
- Narrow recall by sampling unflagged entities
- Re-judge Python samples, and add a judge from another model family
- A corpus with many reviewed rejections
- Repository history as the unit of analysis

Thank you

Questions

Backup

Additional material

Backup: why recall must be weighted

Each sample is half flagged, the population is not. For Java Liskov it holds 574 flagged override pairs and 25,594 unflagged ones.

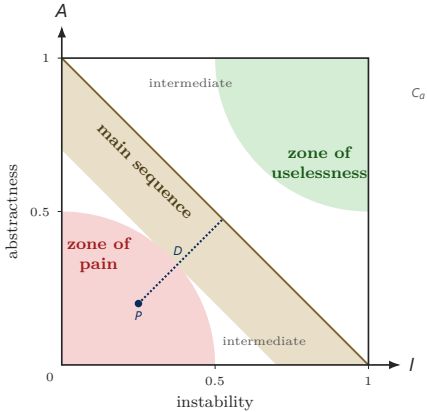
$$R = \frac{TP \cdot N_f/n_f}{TP \cdot N_f/n_f + FN \cdot N_u/n_u}$$

Counting the strata as equally common gives 0.966. Weighting gives **0.386**.

The population is the one the sample was drawn from: production entities, counted once each.

- The intervals are wide because a weighted recall rests on the few violations found in the unflagged stratum (three for Liskov, in 200 rows).
- This is why more labelling would help recall, not precision.

Backup: package zones (descriptive only)









$$I = \frac{C_e}{C_a + C_e}$$

0 when nothing depends outward, 1 when nothing depends on the package

$$A = \frac{N_a}{N_c}$$

share of the package's classes that are abstract (dashed)

$$D = |A + I - 1|$$

distance from the main sequence; recorded for description, not used as a flag

Distance from the main sequence was tested as a change signal and confirmed none of 17 flags, so packages are described by position and never flagged.

Backup: corrected violation rates

Principle	Unit	Java		Python	
		flagged	corrected	flagged	corrected
SRP	class, authored	7.7%	2.7%	8.9%	1.0%
SRP	class, all touched	12.2%	4.3%	11.6%	1.3%
OCP	pull request	6.7%	2.2%	6.4%	2.5%
LSP	pull request	0.4%	0.4%	0.9%	0.5%
ISP	pull request	6.5%	2.2%	0.6%	0.3%
DIP	pull request	4.6%	4.1%	3.5%	3.5%

Corrected = flag

rate multiplied by measured precision. The Java Liskov row is the shipped flag, refused bequest only, whose precision is 1.000.